

Deadline Buffer + Group Work Splitter

Turning stressful deadlines into calm start-by dates & balancing group workloads by teammate availability.

■ Presenter & Stack Glen P React 18 · Vite · Tailwind CSS v4 Supabase PostgreSQL · Render Hosting	■ Deliverables & Score M8A8 Presentation (100 Points) 3–5 Min Video · 8-Slide Deck · 1080×1080 Image AI Attribution: ~25% AI / ~75% Human Effort	■ Production Links Live Application: deadline-buffer.onrender.com GitHub: github.com/qwertyuipaas/deadline-buffer
---	--	--

The 'Due Date Illusion' & Unequal Group Splits

Why standard to-do apps and calendar schedules fail student psychology.

■ The Due Date Illusion

Traditional calendars only display the **final deadline** (e.g. 11:59 PM) and not the **start-by date** (e.g. 10:00 AM on Thursday night).

■ Estimation Blindness

Students know an assignment requires 10+ hours of focus, but they don't know how to allocate that time.

■ The Group Work Dilemma

Group projects split tasks blindly without visibility into **each team member's** availability, leading to unequal workloads and resentment.

■ The Core Opportunity

A smart system that works *backward from the hard deadline*, generates a realistic Start-By Date with a visual safety cushion, and auto-balances tasks based on each member's declared weekly available hours.

The Start-By Engine & Signature Buffer Bar

A deterministic algorithm that replaces cramming with calm, actionable start dates.

■ The Start-By Calculation Math

Assumes a baseline of **2 focused study hours/day**, scaled by priority multipliers:

- **Low Priority:** 1.1× buffer multiplier
- **Medium Priority:** 1.3× buffer multiplier
- **High Priority:** 1.6× buffer multiplier (extra cushion for high stakes)

$\text{bufferDays} = \text{Math.ceil}((\text{hours} / 2) * \text{multiplier})$

$\text{startDate} = \text{deadline} - \text{bufferDays}$

■ The Signature Visual Buffer Bar

A to-scale visual indicator splitting time into two distinct zones:

- **Teal Zone (Safe Buffer):** Days remaining before you need to begin.
- **Amber Zone (Work Window):** Active days reserved for focused work.
- **Red Alert:** Indicates an overdue start date (■ Start today).

Provides an instant 0–100% **Semester Buffer Health Score** on the dashboard.

Primary User Flow & Workload Balancing

From quick task capture to multi-user capacity management and calendar export.

1. Dashboard Overview View real-time Buffer Health Score (0–100%)	2. Quick Task Capture Slide-in Task Drawer opened via N keyboard shortcut	3. Workload Balancer Teammates declare weekly hours (e.g. 10h)	4. Export Hub Generates standard iCalendar (.ics) files compatible with Outlook
--	---	--	--

Full-Stack Architecture & Infrastructure

Built with React 18, Tailwind CSS v4, and PostgreSQL via Supabase.

■ ■ Frontend Layer

- **React 18 SPA + Vite 6:** Fast build and component re-rendering.
- **Tailwind CSS v4:** @theme design tokens, keyframe animations.
- **Custom Hooks:** useProjectData, useTaskForm, useTaskList.
- **Zero-dependency iCalendar generator:** Pure JavaScript.

■ Backend & Database

- **Supabase (Managed PostgreSQL):** Relational database.
- **GoTrue Auth:** JWT session handling with salted bcrypt.
- **Relational Schema:** profiles, projects, project_members.
- **Postgres Triggers:** Automated updated_at synchronization.

■ Hosting & DevOps

- **Render Static Hosting:** Production deployment with SPA rewrite.
- **GitHub Actions Cron:** Scheduled workflow (keep-supabase-alive).
- **Clean Git History:** Meaningful atomic commits tracking features.

■ Collaborative Row Level Security (RLS)

PostgreSQL security policies enforce strict multi-tenant authorization: Project creators maintain full admin CRUD rights; teammates in project_members can view shared projects and update assigned task status without permission to overwrite deadlines or hours.

Engineering Roadblocks & Debugging Solutions

Four real-world bugs caught, traced, and resolved during development.

■ 1. UTC vs Local Timezone Bug

The Error: `toISOString().split('T')[0]` converted dates to UTC, shifting dates back by one day.

The Solution: Wrote custom `toLocalIsoDate()` using local `getFullYear()`, `getMonth()+1`, and `getDate()`.

■ 2. Mobile White Screen Crash

The Error: Desktop Chrome cached older scripts, but visiting on a real phone produced an immediate white screen.

The Solution: Connected mobile remote DevTools and caught `ReferenceError: useEffect is not defined`.

■ 3. Typewriter Layout Shift (CLS)

The Error: Landing page typewriter animated words of varying length, causing the text to jump.

The Solution: Locked container with fixed `min-h-[1.2em]`, block layout, and inserted a placeholder.

■ 4. PostgreSQL RLS Infinite Recursion

The Error: Writing a member policy with an `EXISTS` subquery on `project_members` inside a rule.

The Solution: Broke circular dependency by table aliasing (as `pm`) and scoping queries directly to it.

Security Audit & AI Attribution Breakdown

Exceeding academic integrity standards and course finals guidelines.

■■ Security Checklist (15 / 15 Passed)

- **Secrets Excluded:** .env in .gitignore, only placeholders committed in .env.example.
- **Client Protection:** Service-role secret key kept off client; public anon key used only.
- **Password Security:** Salted cryptographic bcrypt hashing via Supabase Auth.
- **SQL Injection Defense:** Parameterized Supabase client queries (zero raw SQL con
- **XSS Protection:** Automatic React JSX character escaping.
- **Database RLS:** Active on all 4 tables (profiles, projects, project_members, tasks).

■ AI Usage Attribution (~25% AI / ~75% Human)

Course requirement: *at least 20% human-authored (max 80% AI)*. Our project achieved ~75% hu

- **Architecture & App Flow:** 85% Human / 15% AI
- **Frontend Components & UI:** 75% Human / 25% AI
- **Buffer Algorithm & Date Math:** 70% Human / 30% AI
- **PostgreSQL & Security Policies:** 70% Human / 30% AI
- **Documentation & Reflection:** 85% Human / 15% AI

Fully documented in AI-USAGE.md with verification logs and error fixes.

Future Roadmap & Project Evaluation

Next evolution steps for Deadline Buffer.

■ 1. Web Push Notifications

Implement browser push notifications that alert students

■ 2. Multi-Course Gantt Timeline

Build a visual semester Gantt timeline showing overlap

✉ ■ 3. One-Click Teammate Invites

Automate group onboarding via email invitation links using Supabas

■ Conclusion & Submission Links

Thank you for listening! Ready for questions.

- **Live Application:** <https://deadline-buffer.onrender.com/>
- **GitHub Repository:** <https://github.com/qwertyuipaas/deadline-buffer>
- **Design System Specification:** <https://deadline-buffer.onrender.com/design-system/>
- **Midterm Reflection Journal:** <https://deadline-buffer.onrender.com/journal/>